Comparative Analysis of Machine Learning Model Training

Personal Computer (MacBook) vs. Supercomputer (CSC Puhti)

A performance evaluation study using TensorFlow on the MNIST dataset with a Convolutional Neural Network architecture.

Author: Prashant Bhandari

Date: April 2026

Table of Contents

1. Introduction

The rapid advancement of machine learning (ML) has led to an increasing demand for computational resources capable of handling large-scale model training. While high-performance computing (HPC) clusters and supercomputers are often regarded as the gold standard for training complex neural networks, personal computers have also evolved significantly in terms of processing power. This raises an important question: for modest-scale machine learning tasks, does the performance advantage of a supercomputer justify the additional complexity of remote job submission and environment management?

This report presents a comparative analysis of training a Convolutional Neural Network (CNN) on the MNIST handwritten digits dataset using two distinct computational environments: a personal Apple MacBook computer running locally in Visual Studio Code (VSCode), and the CSC Puhti supercomputer, a high-performance computing cluster operated by the Finnish IT Center for Science (CSC). The objective is to evaluate and contrast the training performance, measured primarily in terms of execution time, between these two systems when executing identical model configurations.

By maintaining consistency in the dataset, model architecture, hyperparameters, and deep learning framework across both platforms, this study aims to provide an objective assessment of the practical performance differences between consumer-grade hardware and enterprise-level HPC infrastructure for small to medium-scale machine learning workloads.

2. Methodology

2.1 Dataset Description

The MNIST (Modified National Institute of Standards and Technology) dataset was selected as the benchmark for this comparison. MNIST is a widely recognized dataset in the machine learning community, consisting of 60,000 training images and 10,000 test images of handwritten digits (0-9). Each image is a 28x28 pixel grayscale representation. The relatively small size of MNIST makes it an ideal candidate for rapid experimentation while still providing meaningful performance insights. The dataset was loaded directly via the TensorFlow Keras API, which handles automatic downloading and preprocessing.

2.2 Model Architecture

A Convolutional Neural Network (CNN) was implemented for this study due to its proven effectiveness in image classification tasks. The model architecture consists of the following layers:

- Input Layer: 28x28x1 (grayscale images)
- Convolutional Layer 1: 32 filters, 3x3 kernel, ReLU activation
- Max Pooling Layer 1: 2x2 pool size
- Flatten Layer: converts 2D feature maps to 1D vector
- Dense Layer: 64 units, ReLU activation
- Output Layer: 10 units (one per digit class), softmax activation

2.3 Training Configuration

To ensure a fair comparison, identical training configurations were used on both systems. The model was trained for 5 epochs with a batch size of 32. The Adam optimizer was employed with its default learning rate, and categorical cross-entropy was used as the loss function. The dataset was normalized by scaling pixel values to the range [0, 1] prior to training.

Parameter	Value
Dataset	MNIST (60,000 train / 10,000 test)
Model Architecture	CNN (Conv2D + Dense layers)
Epochs	5
Batch Size	32
Optimizer	Adam (default learning rate)
Loss Function	Categorical Cross-Entropy
Framework	TensorFlow / Keras

2.4 Computational Environments

The experiments were conducted on the following two systems:

System A: Personal Computer (MacBook)A standard Apple MacBook computer was used as the local development environment. The model was executed within Visual Studio Code (VSCode), a popular integrated development environment (IDE) for Python development. Training was performed using the CPU, as no GPU acceleration was configured for the local TensorFlow installation.

System B: CSC Puhti SupercomputerCSC Puhti is a petascale supercomputer located at the Finnish IT Center for Science. It is equipped with Intel Xeon processors and NVIDIA V100 GPUs. For this experiment, an interactive compute node was accessed via SSH, and the TensorFlow module was

loaded using the system's environment module system (module load tensorflow). No GPU resources were explicitly requested for the interactive job, meaning the training also executed on CPU resources within the HPC environment.

3. Code Implementation

The same script was used on both system. On Puhti, the script was executed after running module load tensorflow.

```
"""
MNIST CNN Training Script - Works on both local and Puhti
"""

import tensorflow as tf
import numpy as np
import time
import json
import os

# ========== CONFIGURATION ==========
CONFIG = {
    "epochs": 5,
    "batch_size": 32,
    "model_name": "mnist_cnn"
}

# ========== DATA PREPARATION ==========
def load_and_prepare_data():
    """Load and preprocess MNIST dataset"""
    print("Loading MNIST dataset...")
    (x_train,          y_train),          (x_test,          y_test)          =
tf.keras.datasets.mnist.load_data()

    # Normalize and reshape
    x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
    x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0

    print(f"Training samples: {len(x_train)}")
    print(f"Test samples: {len(x_test)}")
    return (x_train, y_train), (x_test, y_test)
```

```
# ========== MODEL ARCHITECTURE ==========
def create_cnn_model():
    """Create a simple CNN model for MNIST"""
    model = tf.keras.Sequential([
        tf.keras.layers.Conv2D(32,      (3,      3),      activation='relu',
input_shape=(28, 28, 1)),
        tf.keras.layers.MaxPooling2D((2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    model.summary()
    return model

# ========== TRAINING ==========
def train_model():
    """Main training function"""
    print("=" * 50)
    print("STARTING TRAINING")
    print(f"Epochs:         {CONFIG['epochs']},         Batch         size:
{CONFIG['batch_size']}")
    print("=" * 50)

    # Load data
    (x_train, y_train), (x_test, y_test) = load_and_prepare_data()

    # Create model
    model = create_cnn_model()

    # Start timer
    start_time = time.time()

    # Train
```

```
    history = model.fit(
        x_train, y_train,
        epochs=CONFIG['epochs'],
        batch_size=CONFIG['batch_size'],
        validation_data=(x_test, y_test),
        verbose=1
    )

    # End timer
    end_time = time.time()
    training_time = end_time - start_time

    # Evaluate
    test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)

    # Save model
    model.save(f"models/{CONFIG['model_name']}.h5")

    # Collect results
    results = {
        "test_accuracy": float(test_accuracy),
        "test_loss": float(test_loss),
        "training_time_seconds": float(training_time),
        "final_epoch_accuracy": float(history.history['accuracy'][-1]),
        "final_epoch_val_accuracy":  float(history.history['val_accuracy'][-
1]),
        "system_info": {
            "gpu_available":  len(tf.config.list_physical_devices('GPU'))  >
0,
            "gpu_count": len(tf.config.list_physical_devices('GPU')),
            "tensorflow_version": tf.__version__
        }
    }

    # Print summary
    print("\n" + "=" * 50)
    print("TRAINING COMPLETE")
    print(f"Test Accuracy: {test_accuracy:.4f}")
    print(f"Training Time: {training_time:.2f} seconds")
    print(f"GPU Available: {results['system_info']['gpu_available']}")
    print("=" * 50)
```

```
    # Save results to JSON
    with open(f"results_{CONFIG['model_name']}.json", 'w') as f:
        json.dump(results, f, indent=4)

    print(f"Results saved to results_{CONFIG['model_name']}.json")
    print(f"Model saved to models/{CONFIG['model_name']}.h5")

    return results

# ========== MAIN EXECUTION ==========
if __name__ == "__main__":
    # Check GPU availability
    gpus = tf.config.list_physical_devices('GPU')
    if gpus:
        print(f"GPUs detected: {len(gpus)}")
        for gpu in gpus:
            print(f"  - {gpu.name}")
    else:
        print("No GPU detected, running on CPU")

    # Run training
    results = train_model()
```

Listing 1: Python code

4. Results

The primary metric for comparison in this study is the total training time, measured in seconds, required to complete 5 epochs of model training on the MNIST dataset. Both systems successfully trained the CNN model to convergence. The results are summarized in the table below.

System	Training Time (s)	Test Accuracy	Environment
MacBook (VSCode)	25.65	0.9827	Local CPU
CSC Puhti	24.27	0.9836	HPC Interactive Node (CPU)
Difference	1.38	—	—
Relative Improvement	5.4%	—	Puhti faster

Table 2: Training Time Comparison Between MacBook and CSC Puhti

The training times observed were remarkably similar between the two systems. The MacBook completed the 5-epoch training in 25.65 seconds, while the CSC Puhti supercomputer required 24.27 seconds. This represents a difference of approximately 1.38 seconds, or roughly 5.4% in relative terms, with the supercomputer achieving a modestly faster time.

Model accuracy on the test set reached 0.9827 on the MacBook and 0.9836 on the Puhti system, confirming that both environments produced functionally equivalent models in terms of predictive performance.

5. Discussion

The results of this comparative study reveal several important insights regarding the performance characteristics of personal computers versus supercomputers for modest-scale machine learning tasks.

5.1 Minimal Performance Difference

The most striking observation is the minimal difference in training time between the MacBook and the CSC Puhti supercomputer. With only a 5.4% performance advantage, the supercomputer did not demonstrate the dramatic speedup that might be expected from HPC infrastructure. This can be attributed to several factors:

- Dataset Size: MNIST is a relatively small dataset. The computational overhead of data loading and preprocessing constitutes a significant portion of the total training time, and this overhead is similar across both systems. The true advantage of HPC systems becomes apparent with larger datasets (e.g., ImageNet, COCO) where I/O bottlenecks and memory bandwidth limitations become more pronounced.

- CPU-Only Execution: Neither system utilized GPU acceleration for this experiment. Modern supercomputers derive much of their performance advantage from massively parallel GPU architectures. Without leveraging the NVIDIA V100 GPUs available on Puhti, the comparison effectively pits the MacBook's CPU against Puhti's CPU cores, which are both modern Intel architectures with comparable per-core performance for single-threaded or lightly threaded workloads.

- Model Complexity: The CNN architecture used in this study is relatively shallow, consisting of only two convolutional blocks. More complex architectures (e.g., ResNet, Transformer-based models) with millions or billions of parameters would better exploit the parallel processing capabilities of HPC systems.

5.2 Memory Management Considerations

A notable difference between the two systems lies in their memory management approaches. The MacBook operates with a unified memory architecture, where the operating system dynamically manages memory allocation between the CPU and integrated graphics. In contrast, the CSC Puhti supercomputer employs a distributed memory model with explicit resource allocation through the Slurm workload manager.

On Puhti, memory allocation is strictly enforced based on the job submission parameters. The interactive node session had predefined memory limits, which, while sufficient for the MNIST workload, would require careful tuning for larger models or batch sizes. The MacBook, while more flexible in memory allocation, may experience performance degradation due to background processes and operating system overhead that are not present in the controlled HPC environment.

5.3 Practical Implications

For researchers and practitioners working with small to medium-scale datasets similar to MNIST, these results suggest that a modern personal computer is entirely adequate for model development and prototyping. The overhead of connecting to a remote supercomputer, managing environment modules, and navigating job submission systems may not be justified when the performance gain is marginal.

However, for large-scale experiments involving massive datasets, distributed training across multiple nodes, or hyperparameter search over many configurations, the HPC environment offers significant advantages in terms of scalability, dedicated resources, and the ability to run multiple experiments in parallel.

6. Challenges Faced

Several technical challenges were encountered during the execution of this comparative study, particularly when working with the CSC Puhti supercomputer environment. These challenges and their resolutions are documented below.

6.1 Module Loading and Environment Configuration

Unlike a local Python environment where dependencies are typically managed via pip or conda, the Puhti supercomputer uses a module system for loading pre-installed software packages. The initial attempt to use pip-installed TensorFlow resulted in compatibility errors due to mismatched library versions. The solution was to use the module load tensorflow command, which loads a pre-configured, optimized TensorFlow environment with all necessary dependencies.

6.2 Interactive Node Limitations

The experiment on Puhti was conducted using an interactive compute node (sinteractive) rather than a batch job submission. Interactive nodes are subject to time limits and shared resource allocation. Additionally, GPU resources were not available on the interactive node without explicit reservation. Attempting to request GPU allocation for the interactive session would require additional parameters (e.g., --gres=gpu:v100:1) and may be subject to queue wait times depending on system load.

6.3 Data Transfer and File System Access

While MNIST is automatically downloaded by TensorFlow, working with custom datasets on Puhti requires careful management of file paths across the different storage systems (home directory, scratch, and project directories). The Allas object storage service and the module load allas command are available for persistent data storage, but they introduce additional complexity compared to local file system access.

6.4 Network Latency and SSH Connectivity

Remote access to Puhti requires a stable SSH connection, which can be affected by network latency and institutional firewall configurations. Disconnections during interactive sessions can result in lost work, necessitating the use of terminal multiplexers such as screen or tmux to maintain persistent sessions.

7. Conclusion

This report presented a comparative evaluation of machine learning model training on two distinct computational platforms: a personal Apple MacBook and the CSC Puhti supercomputer. Using a Convolutional Neural Network trained on the MNIST dataset as the benchmark, the study found that the difference in training time between the two systems was minimal, with the supercomputer achieving only a 5.4% reduction in execution time (24.27 seconds vs. 25.65 seconds).

The key findings of this study are as follows:

0. For small-scale datasets such as MNIST, the performance advantage of a supercomputer over a modern personal computer is marginal when both systems are executing on CPU resources.

1. The true benefits of HPC infrastructure, including the CSC Puhti supercomputer, are realized through GPU acceleration, distributed computing, and the ability to execute large-scale parallel experiments.

2. Working with HPC systems introduces additional complexity related to environment management, job scheduling, and remote access, which may not be justified for simple prototyping tasks.

3. Memory management and resource allocation are more explicit and controlled on HPC systems, which can be advantageous for reproducible research but requires additional setup effort.

In conclusion, while supercomputers remain indispensable for large-scale machine learning research, this study demonstrates that personal computers are fully capable of handling small to medium-scale deep learning tasks efficiently. Researchers should carefully evaluate the scale of their experiments and the availability of GPU resources when deciding between local development and HPC deployment